MEng Project

Department of Computing

Imperial College London of Science, Technology and Medicine

Ginseng: Staged Metaprogramming Graphics Library

Author:
Joshua Walker

Supervisor:
Jamie Willis

Second Marker:
Tom Crossland

Submitted in partial fulfillment of the requirements for the Master's degree in Computing of Imperial College London

January 22, 2026

Abstract

Abstract here.

Acknowledgments

To Jamie and Tom, my thanks and apologies for reading all of this.

Contents

1 Introduction **1**

1.1 Problem 1

1.2 Project Plan 1

1.2.1 Project Timeline 2

2 Background **3**

2.1 Domain Specific Languages 3

2.1.1 Deep and Shallow Embeddings 3

2.1.2 Bindings 4

2.2 Staged Metaprogramming 4

2.3 Computer Graphics 5

2.3.1 Graphics Pipeline 5

3 Related Work **9**

3.1 Domain Specific Languages 9

3.2 Graphics APIs and Libraries 9

3.2.1 OpenGL 9

3.2.2 Vulkan 15

3.2.3 DirectX 15

3.2.4 Pixar RenderMan 15

3.2.5 Haskell GPipe 15

4 Evaluation **18**

4.1 Benchmarking 18

4.2 User Testing 18

5 Declarations **19**

5.1 Use of Generative AI 19

5.2 Ethical Considerations 19

5.3 Sustainability 19

Chapter 1

Introduction

1.1 Problem

Modern graphics libraries are trending towards providing lower level APIs for more exacting control over the GPU hardware with the advent of photorealistic graphics and need for highly performant systems.[1] The skill required by graphics programmers is steadily increasing, outstripping any modernisation efforts made for the development environment. Programming concerns are not separated; without domain-specific knowledge being utilised to make assumptions about the environment, a developer has full control over the hardware, at the cost of reimplementing many of the programs classically handled by operating software drivers.

1.2 Project Plan

To reduce the cognitive load upon technical artists, I will define a domain-specific language to simplify the classical model of a graphics API; this involves making assumptions about the domain in order to abstract out lower-level functionality – for example, this may mean limiting the environment to utilise 2D primitives, simplifying shader definition or rendering static scenes without the support of input handling. This is to build a library used for scene synthesis and as such may require dismissing concerns related to more advanced graphics rendering techniques such as animation.

Developing a graphics library in this way allows me to bake in type-safe guarantees with the design of my domain-specific language; conversely, most modern graphics APIs are designed using an unsafe and unprincipled imperative state machine, which requires careful consideration and frustrates debugging attempts. This DSL has the ability to provide a compromise between imperative paradigms and declarative one – abstracting out state dynamics allows for declarative scene declaration, whilst providing access to lower-level imperative methods to define dynamics of the underlying renderer.

An interpreter is built upon the domain specific language, expanding terms into their constituent graphics API bindings. Performance concerns may arise due to exhaustive process of expanding each declared domain-specific primitive composing each scene object into their corresponding underlying graphics primitive. To alleviate this bottleneck, staged metaprogramming can be employed to abstract large sections of

interpreted code into a tailored elimination of redundancies and eliminate the overhead of pipelined interpretation. As the scene is statically known, a large degree of knowledge pertaining to the final render is available to the interpreter, which can be used to perform optimising techniques that are infeasible when operating within an state-driven imperative language – i.e., wholesale vertex batching, reduction of drawing mode modifications, etc.

Some further improvements may then expand upon the domain specific language – as a proof of concept, it may initially be very simple or restrictive. To allow programmers a larger degree of freedom, I could extend the DSL, for example, from 2D to encapsulate 3D objects or support animation or complex shading.

1.2.1 Project Timeline

A basic timeline for the project will be as follows:

- **December – Early Jan**: Research low-level graphics APIs, example domain specific languages and staged metaprogramming.
- **Jan – Early Feb**: Develop DSL representations of 2D geometry; define combinatorial logic allowing them to be composed in relation to their position and geometry.
- **Feb – March**: Develop an interpreter to convert the embedded DSL into low-level graphics API code.
- **March**: Refine DSL representation.
- **Late March – April**: Begin staging common code, transforming interpreter into a compiler.
- **Late April – June**: Extend DSL to provide support for complex shading, 3D geometry, animation, etc.,.
- **April – June**: Document project achievements in final draft of report.

Chapter 2

Background

This section aims to give sufficient background on principled staged metaprogramming and computer graphics in order to provide foundations for understanding methods discussed in this thesis.

2.1 Domain Specific Languages

General-purpose programming languages like Scala, Haskell, and C are designed for wide-ranging applications; this may have the negative side-effect of bringing into focus many solutions to a single problem. The resulting tension motivates the need for a *domain specific language* (DSL), in which a language is specialised to a paradigm, such as SQL for database management, MATLAB for matrix programming, YACC for developing parsers, etc.,. These languages have programmatical structures designed for this application, with domain-specific assumptions being made to facilitate development.

Domain-specific languages take a variety of semantic and syntactical forms; they can be divided into two categories – *standalone* or *embedded*. Standalone DSLs have a wide degree of freedom to redefine their syntax and semantics as they are hosted externally with a newly defined toolkit (compiler, debugger, etc.). Embedded domain specific languages are hosted *within a GPL*; they can be thought of as a set of definitions within the host language, or a stylized use of the language for a domain-specific purpose.[2]

2.1.1 Deep and Shallow Embeddings

There is no standard method of embedding a DSL; however, the notion of the depth of an embedding is used to further categorize embedded DSLs into *shallow* and *deep* embeddings. With a *shallow embedding*, terms within the DSL are simply constructed as an implementation of their semantics; conversely, the semantics of a deeply embedded DSL are affected through the evaluation of an internal abstract syntax tree (AST), subject to optimisation.[3]

2.1.2 Bindings

A *binding* is an application programming interface (API) that provides access to a library from within its non-native programming language. Bindings can either be *direct* – i.e., a syntactic mapping from one language to another, preserving naming schemes and structures – or *abstract*, in which the semantics of the library are best replicated in the target language.[4]

A common use-case for bindings is accessing the wealth of C libraries; the C header system provides publicly accessible functions and type definitions, and as such is a valuable resource automating binding generation. As noted in the following sections, many graphics libraries are specified using C as a standard; in this case, bindings will be required to access the library within Scala.

2.2 Staged Metaprogramming

Metaprogramming is a method of manipulating code as a first-class value: code is constructed using regular language syntax, enclosed within the *quote* syntax ([| ... |] in Haskell; `'{ ... }` in Scala). The converse operator is *splice*, which evaluates quoted code and inserts the result into the program ($ in both Haskell and Scala).

Staging is a method in which the programmer identifies statically-known sections of a program and utilises metaprogramming to perform the computation ahead of runtime. The traditional example of two-stage metaprogramming and its utilisation is in redefining the `power n x` function for calculating x^n:

```
power :: Nat -> (Int -> Int)
power 0 = \x -> 1
power n = \x -> x * power (n - 1) x
```

Listing 2.1: Classical power function[5]

```
power' :: Nat -> Code (Int -> Int)
power' 0 = [| \x -> 1 |]
power' n = [| \x -> x * $(power' (n
    ↳- 1)) x |]
```

Listing 2.2: Staged power function[5]

As such, when executed, the staged power function evaluates to the following code:

```
power5 = $(power' 5)
       = $([| \x -> x * x * x * x * x * 1 |])
       = \x -> x * x * x * x * x * 1
```

Listing 2.3: Evaluated staged power function[5]

Multiple stages arise when higher-level abstractions cannot be eliminated using a single application of the quote and splice primitives; to expose larger degrees of static information, a program can be converted into a syntactic representation at an initial stage. At later stages, this representation is evaluated using the wider syntactic context to infer greater meaning.[6]

2.3 Computer Graphics

Computer graphics is an an area of computer science focused on the technical aspects involved in ensuring visual content such as art, image and film is meaningfully visible to a consumer. Sub-fields include image processing, rendering and visual content manipulation.

This thesis will focus on the rendering of images to screen, typically in the example of personal computers and games consoles responsible for displaying high-definition scenes synthesized by technical artists.

2.3.1 Graphics Pipeline

The classical model of a graphics render pipeline consists of three major stages: **application**, **geometry**, **rasterisation**. Figure 2.1 is an example of render pipeline.[7]

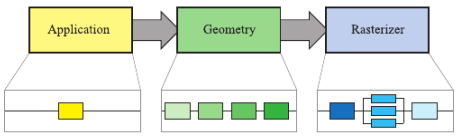

Figure 2.1: The graphics render pipeline divided into major stages[7]

Application

This stage is driven by the underlying application; executed on a general purpose CPU, it modifies the original scene according to software managed by a developer with full control over the implementation. Some operations typically performed during this stage include physics simulation, collision detection, input handling, etc.

At the end of this application stage, the scene data is forwarded to the geometry stage in the form of rendering primitives: triangles, points, lines, etc., that constitute the final scene.

Geometry

The geometry stage performs the majority of per-polygon and per-vertex operations; this stage can be further delineated into the **model and view transformation**, **vertex shading**, **projection**, **clipping** and **screen mapping** stages as shown in Figure 2.2.

Figure 2.2: The geometry stage divided into sub-stages[7]

Model and View Transform Scene data must be understood in relation to a user-defined virtual camera, indicating the position and viewing direction from which the scene is to be rendered. This gives graphics primitives and their position meaning.

Each model resides in its own *model space*, untransformed and without relation to a viewer or the scene. Each model is associated with a *model transform* matrix to reposition and orientate the model within the scene at *world space coordinates*. Many different model transformations can be applied to the same model, resulting in differing copies (called *instances*) of the model in the same scene.

To perform simpler clipping and projection operations at later stages within the geometry pipeline, the scene is transformed by the *view transform* matrix such that the viewer is positioned at the origin, orientated such that it aims along the $-z$ axis (this is by convention – some texts prefer to use the $+z$ axis). This view transform is applied to all models and the camera, transforming the world space into the *camera space*.

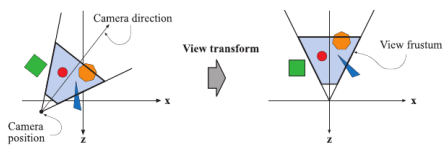

Figure 2.3: The view transformation[7]

Vertex Shading To produce a realistic scene, models are illuminated by light sources placed in the scene. At each vertex, the illumination from the sources is computed in accordance with the material values ascribed to the model. The material determines the reflectivity of the model, colour, etc., and can be used in conjunction with texture mapping to create photorealistic models. These computed shaded vertex values are sent to the per-pixel rasterisation stage for interpolation.

Projection The view space is then transformed into a *canonical view volume* – there are two commonly used projections: *perspective* and *orthographic*.

The **orthographic projection** transforms the camera space into a unit cube, with extremities at $(1, 1, 1)$ and $(-1, -1, -1)$. This projection maintains parallelism of lives throughout the transformation.

The **perspective projection** is more complex, transforming the camera space into a *frustum* – a truncated pyramid with a rectangular base. This projection does not maintain parallel lines, mimicking the change in size of objects further from the camera by scaling them accordingly.

Regardless of the projection used, this is a distinct mathematical operation from transformation as (despite maintaining a z-axis temporarily for rasterizing) the z coordinate is disregarded and the model is projected from three to two dimensions.

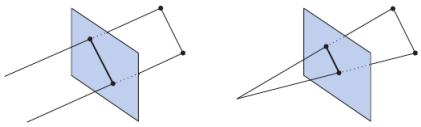

Figure 2.4: Orthographic projection (left) compared with perspective projection (right)[7]

Clipping For performance, only primitives that are within the visible volume defined by the previous projection stage need to be passed to the rasteriser. Primitives contained within the viewing volume are unmodified; primitives that are fully occluded from the view are removed; primitives that are partially within the view volume require *clipping*. This process involves computing the intersection of the view volume boundaries and the line between vertices and replacing the clipped vertex with intersection point. This utilises the benefit of the consistent transformation to a unit view volume to standardise the clipping operation.

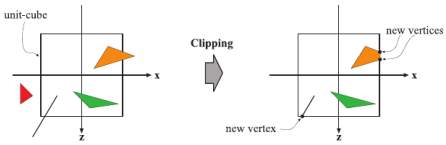

Figure 2.5: The clipping operation performed on a sample scene[7]

Screen Mapping The clipped primitives still maintain their three-dimensional form despite the previous projection stage. The x and y coordinates are transformed into *screen coordinates*; the x, y and z coordinates form the *window coordinates*. By convention, the screen coordinate system has the origin $(0, 0)$ at the *top-left* corner. To map the unit view volume to the screen, a translation followed by a scaling operation is performed.

Rasterisation

The rasteriser samples continuous primitives into discrete pixels on the screen. During this stage, raster points are called *fragments* – this distinguishes between a single pixel on the screen and multiple pixel candidates within the frame buffer. The constituent stages of the rasteriser can be seen in Figure 2.6

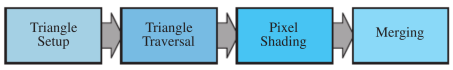

Figure 2.6: The rasterise stage divided into sub-stages[7]

Each fragment contained within the triangle has values computed by interpolation between its three defined vertices using *Barycentric* coordinates. *Fragment shading* or pixel shading computation is performed using this interpolated fragment data – this may include *texturing*, a process in which (typically) two-dimensional images are 'glued' onto the surface of an object.

To render overlapping polygons, the process of *z-buffering* is performed in order to ensure the visibility of the closest polygons to the viewer. This is the reason for screen space primitives maintain their three-dimensional form; during z-buffering, these z-coordinates determine the closest fragments. The algorithm simply overwrites existing pixel values with any fragments with a smaller z-coordinate value – partially transparent primitives must be rendered after all opaque primitives in a back-to-front order.

This scan conversion operation transforms each fragment (or pixel candidate) into a single pixel. To prevent gradual rendering of primitives, once the image is completely rastered, it is copied to the *frame buffer*, a visible area of memory to be displayed on screen.

Chapter 3

Related Work

This section studies graphical APIs and libraries to perform rendering and utilising staged metaprogramming

3.1 Domain Specific Languages

There exist many examples of domain specific languages – some relevant to the development of a graphical DSL include: the *Geometric Constructions Language* (GCL)[8] for explicit description of constructions in the Euclidean plane for use in providing mathematical illustrations and automating geometric proofs ; *Selos*[9] which demonstrates the use of staged metaprogramming techniques within the graphical domain to generate efficient shaders with strong design fundamentals; and a modular declarative JSON-embedded domain-specific language for the specification of 3D visualisation tools.[10]

3.2 Graphics APIs and Libraries

Typical graphics APIs and libraries can be categorised as either *imperative* or *declarative*; i.e., the program either describes *how* a scene is rendered, or *what* should be rendered in the scene.

This section also makes a further distinction between theoretical API specifications supported by hardware vendors, and practical graphics libraries implemented in code on top of aforementioned APIs.

The following are representative of the industry standard of graphics APIs and libraries; these studies will inform the design methodology used and, furthermore, which should be used as the compile target underscoring the DSL.

3.2.1 OpenGL

OpenGL (Open Graphics Library) is a cross-platform specification for a programming interface for the development of 3D graphical applications, allowing developers control over the graphics pipeline as processed by graphics hardware. It is supported by hardware vendors who may define further extensions to the API, and as such is extremely portable – it is the de facto standard for games development on non-Microsoft platforms such as Sony and Nintendo consoles.[11]

OpenGL is an imperative, procedural graphics API; a technical artist will specify the steps necessary to render a scene, not its content. It is designed as a *state machine*, in which drawing commands are parameterised by the currently defined state. Users are not required to consistently redefine variables such as vertex colour or scene lighting as they are persistent until changed. This also has a performance benefit; changes in drawing mode are computationally expensive when repeatedly performed during rendering.

The specification consists of approximately 250 commands used for representing a scene, from defining primitives and their positions to the texturing methods used. It does not include functions to support window management, user interaction or file input/output.

OpenGL is no longer in active development; Khronos Group have begun developing its successor *Vulkan*, which aims to provide developers with more direct control over the GPU.

Primitives and Data Types

OpenGL defines its own data types to ensure they are portable. Each data type begins with the `GL` prefix and is typically followed by a familiar C data type. Some useful examples are `GLboolean`, `GLuint`, `GLenum`.[11][12]

OpenGL primitives are defined in batches of vertices held in a *vertex buffer object*. The primitive rendered using these vertices is specified using one of the following primitive modes:

- `GL_POINTS` – a single vertex on screen with a set diameter, of which the default is 1 pixel.
- `GL_LINES` – a pair of vertices defines a line segment, hence a batch should contain an equal number of vertices. These lines are 1 pixel in width by default.
- `GL_LINE_STRIP` – all vertices are connected in series using a 1 pixel wide line.
- `GL_LINE_LOOP` – all vertices are connected in series, with an additional line connecting the last and first vertices.
- `GL_TRIANGLE` – each 3 vertices in the batch defines a new triangle.
- `GL_TRIANGLE_STRIP` – the first 3 vertices define a triangle, with each subsequent vertex defining an additional triangle formed using the previous 2 vertices.
- `GL_TRIANGLE_FAN` – the first 3 vertices define a triangle, with each subsequent vertex defining an additional triangle formed using the first and previous vertices.

Triangles are the only polygons OpenGL supports; they are the simplest possible polygon and utilised extensively by the rasteriser. When defining a triangle, OpenGL uses the *winding* (i.e., specified order and relative position) of vertices defines its normal: if the vertices of a triangle are defined *counterclockwise*, it is front-facing; otherwise is is backwards-facing. This is important for shading, as different materials may be assigned to the different faces. Textures are also reversed when applied to a back-facing triangle. Note that when drawing triangle strips and fans, the winding of the *first* triangle is used to determine the whether the primitive is front-facing; when drawing independent triangles, the winding is calculated individually for each triangle.

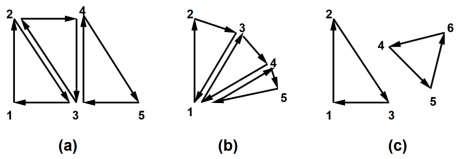

Figure 3.1: (a) Triangle strip. (b) Triangle fan. (c) Independent triangles[12]

GLSL

GLSL (OpenGL Shading Language) is a C-style language used by OpenGL to define vertex, fragment and geometry shaders. It is injected into OpenGL using the `CreateShader`, `ShaderSource` and `ShaderCompile` commands.[12]

Shaders are executed during the render pipeline and define variables with the `in` and `out` keywords for intercommunication between stages – for example, a vertex colour is typically defined in the vertex shader and consumed by the fragment shader. As the fragment shading is performed after rasterisation, inputted values are interpolated from any variables output by the vertex shader.

GLUT

GLUT (OpenGL Utility Toolkit) is the commonly used library for platform-agnostic GUI features such as popup menus and window management. It also support user input such as keyboard and joystick support. This helps developers avoid specialising in differing platform-specific GUI stacks. The open-source alternative *freeglut* is widely used.[12]

OpenGL Extensions

One of the major benefits of OpenGL is its wide compatibility; as hardware developers continue to innovate, they may contribute to the OpenGL API through its extension mechanism. This allows OpenGL to maintain a standard whilst supporting advanced functionality. Each extension method is specified with a prefix, corresponding to the developer; common extension prefixes are `SGI_` (Silicon Graphics),

WGL_ (Microsoft) and NV_ (NVIDIA).[12]

GLEW (OpenGL Extension Wrangler Library) manages OpenGL extensions, initialising the required function pointers and included required constants, typedefs, etc. as defined by the extension.

GLAD is an alternative to GLEW which generates the required loader files from the specified required extensions – it is not able to detect extensions used at runtime. This has the benefit of slightly faster compile times and runtime initialization.[13]

'Hello Triangle'

The following program is a program using OpenGL to render the classical *'Hello Triangle'* example scene in which a 2D triangle with Red, Green and Blue vertices is shaded using values interpolated by the rasteriser.[14]

```
#include <glad/glad.h>
#include <GLFW/glfw3.h>
#include <stdio.h>

int main( void ) {
    // Start OpenGL context and OS window using the GLFW helper library.
    if ( !glfwInit() ) {
        fprintf( stderr, "ERROR: could not start GLFW3.\n" );
        return 1;
    }

    // Request an OpenGL 4.1, core, context from GLFW.
    glfwWindowHint( GLFW_CONTEXT_VERSION_MAJOR, 4 );
    glfwWindowHint( GLFW_CONTEXT_VERSION_MINOR, 1 );
    glfwWindowHint( GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE );
    glfwWindowHint( GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE );

    // Create a window on the operating system, then tie the OpenGL
        ↳ context to it.
    GLFWwindow* window = glfwCreateWindow( 800, 600, "Hello Triangle",
        ↳ NULL, NULL );
    if ( !window ) {
        fprintf( stderr, "ERROR: Could not open window with GLFW3.\n" );
        glfwTerminate();
        return 1;
    }

    glfwMakeContextCurrent( window );

    // Start Glad, so we can call OpenGL functions.
    int version_glad = gladLoadGL();
    if ( version_glad == 0 ) {
        fprintf( stderr, "ERROR: Failed to initialize OpenGL context.\n"
            ↳ );
        return 1;
    }
```

```
// Try to call some OpenGL functions, and print some more version
    ↳ info.
printf( "Renderer: %s.\n", glGetString( GL_RENDERER ) );
printf( "OpenGL version supported %s.\n", glGetString( GL_VERSION ) )
    ↳ ;

const char* vertex_shader =
    "#version 410 core\n"
    "in vec3 vp;"
    "out vec4 vertex_color;"
    "vec4 colors[3] = vec4[3]("
    "    vec4(1, 0, 0, 1),"
    "    vec4(0, 1, 0, 1),"
    "    vec4(0, 0, 1, 1)"
    ");"
    "void main() {"
    "  gl_Position = vec4( vp, 1.0 );"
    "  vertex_color = colors[gl_VertexID];"
    "}";

const char* fragment_shader =
    "#version 410 core\n"
    "in vec4 vertex_color;"
    "out vec4 frag_color;"
    "void main() {"
    "  frag_color = vertex_color;"
    "}";

GLuint vs = glCreateShader( GL_VERTEX_SHADER );
glShaderSource( vs, 1, &vertex_shader, NULL );
glCompileShader( vs );

GLuint fs = glCreateShader( GL_FRAGMENT_SHADER );
glShaderSource( fs, 1, &fragment_shader, NULL );
glCompileShader( fs );

GLuint shader_program = glCreateProgram();
glAttachShader( shader_program, fs );
glAttachShader( shader_program, vs );
glLinkProgram( shader_program );


float points[] = {
    -1.0f, -1.0f, 0.0f, // x,y,z of first point.
     1.0f, -1.0f, 0.0f, // x,y,z of second point.
     0.0f,  1.0f, 0.0f  // x,y,z of third point.
};

GLuint vbo = 0;
glGenBuffers( 1, &vbo );
glBindBuffer( GL_ARRAY_BUFFER, vbo );
glBufferData( GL_ARRAY_BUFFER, 9 * sizeof( float ), points,
    ↳ GL_STATIC_DRAW );

GLuint vao = 0;
```

```
    glGenVertexArrays( 1, &vao );
    glBindVertexArray( vao );
    glEnableVertexAttribArray( 0 );
    glBindBuffer( GL_ARRAY_BUFFER, vbo );
    glVertexAttribPointer( 0, 3, GL_FLOAT, GL_FALSE, 0, NULL );


    while ( !glfwWindowShouldClose( window ) ) {
        // Update window events.
        glfwPollEvents();

        // Wipe the drawing surface clear.
        glClear( GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT );

        // Put the shader program, and the VAO, in focus in OpenGL's
            ↳ state machine.
        glUseProgram( shader_program );
        glBindVertexArray( vao );

        // Draw points 0-3 from the currently bound VAO with current in-
            ↳ use shader.
        glDrawArrays( GL_TRIANGLES, 0, 3 );

        // Put the stuff we've been drawing onto the visible area.
        glfwSwapBuffers( window );
    }

    // Close OpenGL window, context, and any other GLFW resources.
    glfwTerminate();
    return 0;
}
```

Listing 3.1: OpenGL *'Hello Triangle'* Program[14]

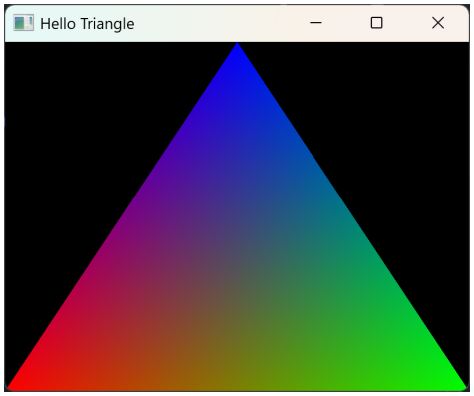

Figure 3.2: Output of the OpenGL *'Hello Triangle'* Program

Many of the difficulties of OpenGL programming can be noted even in this small example: shader type-safety is an impossibility using a `string` representation; initialising a vertex buffer requires several boilerplate OpenGL command calls; the consistent use of constants, enums and bit-flags to modify OpenGL state can be obtuse; and despite utilising the GLAD library, window initialisation is still extensive

and verbose.

3.2.2 Vulkan

Vulkan is the new low-level graphics API currently in development by the Khronos Group, responsible for the development of OpenGL. It aims to improve upon OpenGL, with a focus on minimising the level of abstraction required by the device driver, bringing performance benefits such as lower CPU overhead and memory footprint – as such, it is more comparable to OpenGL ES, the subset of OpenGL designed for accelerating graphical hardware within embedded systems.[1]

Vulkan also utilises more modern ideas of graphics APIs: adopting object-based state management over than monolithic global state; using a multi-threaded rendering model; and separating resources from their underlying physical memory.

This lower-level specification pushes greater responsibility onto the developers, who will be required to implement much of the memory allocation, sychronisation and thread management previous provided by a more complex GPU driver. In this way, Vulkan can be described as more of a GPU API for more advanced graphics projects with a greater need for optimisation.

3.2.3 DirectX

The Direct3D graphics API and its accompanying libraries providing support for user input, window management, audio playback, etc., are collectively known as the *DirectX* suite.

3.2.4 Pixar RenderMan

In contrast to the above imperative examples, the RenderMan Interface Specification (RISpec) developed by Pixar is an open-source declarative API; technical artists describe a three-dimensional scene and utilise separate rendering software which implements the RenderMan Inteface to produce the scene. This compatibility allows artists to use a wide range of industry-standard renderers such as Maya, Houdini and Blender.[15,16,17]

3.2.5 Haskell GPipe

GPipe is a Haskell library providing a functional API for OpenGL 3.3 with the emphasis on type-safe guarantees. As middleware, GPipe subsumes existing OpenGL bindings with its API to avoid enforcing clunky management of the imperative OpenGL state machine within Haskell's monadic system. As noted in the developer's blog post, maintaining these required invariants could lead to frustrating debugging sessions whilst trying to diagnose runtime crashes.[18]

GPipe provides a strong example of a functional graphics library reducing the complexity of OpenGL and its rendering mechanisms. Below is the *'Hello Triangle'* example rendered using the GPipe library:

```
{-# LANGUAGE ScopedTypeVariables, PackageImports, TypeFamilies #-}
module Main where

import Graphics.GPipe
import qualified "GPipe-GLFW" Graphics.GPipe.Context.GLFW as GLFW
import Control.Monad (unless)

main :: IO ()
main =
  runContextT GLFW.defaultHandleConfig $ do
    win <- newWindow (WindowFormatColor RGB8) (GLFW.WindowConfig 800 600
          ↳ "Hello Triangle" Nothing [] Nothing)

    -- Buffer of V4 positions and V3 colours
    vertexBuffer :: Buffer os (B4 Float, B3 Float) <- newBuffer 3
    writeBuffer vertexBuffer 0 [(V4 (-1) (-1) 0 1, V3 1 0 0),
                                (V4   1  (-1) 0 1, V3 0 1 0),
                                (V4   0    1  0 1, V3 0 0 1)]

    shader <- compileShader $ do
      primitiveStream <- toPrimitiveStream id
      fragmentStream <- rasterize (const (FrontAndBack, ViewPort (V2 0 0)
            ↳ (V2 500 500), DepthRange 0 1)) primitiveStream
      drawWindowColor (const (win, ContextColorOption NoBlending (V3 True
            ↳ True True))) fragmentStream

    loop vertexBuffer shader win

loop vertexBuffer shader win = do
  render $ do
    clearWindowColor win (V3 0 0 0)
    vertexArray <- newVertexArray vertexBuffer
    let primitiveArray = toPrimitiveArray TriangleList vertexArray
    shader primitiveArray
  swapWindowBuffers win

  closeRequested <- GLFW.windowShouldClose win
  unless (closeRequested == Just True) $
    loop vertexBuffer shader win
```

Listing 3.2: GPipe *'Hello Triangle'* Program[19]

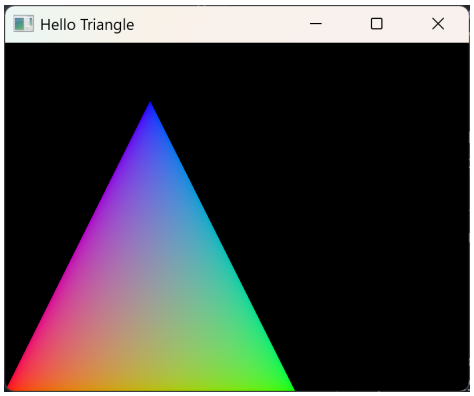

Figure 3.3: Output of the GPipe *'Hello Triangle'* Program

Note that in comparison to the example OpenGL code in Listing 3.1, the GPipe example abstracts much of the typical OpenGL boilerplate and provides an in-built functional and type-safe shader system, unlike the competing Haskell graphics library *luminance* which utilises GLSL.

It should also be noted that GPipe maintains the OpenGL mechanism of vertex buffers; according to the developer Tobias, this decision was taken for the release of version 2.0 in response to the "major flaw" of version 1., which automatically converted the standard Haskell lists into vertex buffers and resulted in memory leaks. This alternative provides the user access to the original lower level functionality OpenGL expects at a detriment to the conciseness of the program.[19]

Chapter 4

Evaluation

The purpose of this thesis is to develop a usable alternative to standard graphics APIs; however, it is infeasible to provide a consistent staged performance benefit without making assumptions about the target domain. During evaluation, I must ensure that the library conforms to user's expectations whilst delivering performance and usability benefits, without the ambition to wholesale replace the inherent performance benefit manipulating direct hardware using low-level APIs.

4.1 Benchmarking

To quantify the performance benefits wrought by staged metaprogramming techniques, I will benchmark the library against common graphics API and library alternatives generating similarly rendered sample scenes, chosen arbitrarily with a wide range of rendering techniques displayed. It is important to compare both the compilation time and runtime performance, including memory used, presence of any memory leaks, error handling and time taken to render. It is important to note that in comparison to the graphics API which the library will compile to, there may be little to no performance benefit, and will likely have an increased compilation time due to the additional staging. The greater benefits will likely be evident when viewed in comparison to competing graphics libraries which perform interpreting operations to generate API code.

4.2 User Testing

To evaluate qualitative criteria such as ease of use, it is important to aggregate user feedback; in the absence of large groups of graphics programmers to survey, it may be necessary to approximate usability using program complexity metrics such as lines of code, nesting depth, and cyclomatic complexity.[20]

Chapter 5

Declarations

The following declarations are made regarding this thesis:

5.1 Use of Generative AI

Generative AI tools were not used at any point during this project; I remain ethically opposed to its use and prevalance.

5.2 Ethical Considerations

Whilst this project does not raise any immediate ethical concerns, as a general-purpose graphics library, it has many applications – some of which may require further ethical consideration, such as military usage or the generation of inappropriate images. This can be addressed through the use of a licensing agreement to ensure liability remains with the user.

5.3 Sustainability

This project provides an interface for GPU use – any lack of optimisation in the tool may require increased computation and power consumption, which must be avoided to ensure a sustainable and usable graphics API.

Bibliography

1. KhronosGroup. Vulkan Basics - Vulkan-Samples. GitHub. 2023 Sep 18. Available from: https://github.com/KhronosGroup/Vulkan-Samples/blob/main/samples/vulkan_basics.adoc [Accessed on: 2026 Jan 21]
2. Martin Fowler and Rebecca Parsons. Domain Specific Languages. Addison-Wesley Professional, 2010 Sep. 640 pp. Available from: https://learning.oreilly.com/library/view/domain-specific-languages/9780132107549/ [Accessed on: 2026 Jan 22]
3. Gibbons J and Wu N. Folding domain-specific languages: deep and shallow embeddings (functional Pearl). *Proceedings of the 19th ACM SIGPLAN international conference on Functional programming*. ICFP'14: ACM SIGPLAN International Conference on Functional Programming. Gothenburg Sweden: ACM, 2014 Aug 19:339–47. DOI: 10.1145/2628136.2628138. Available from: https://dl.acm.org/doi/10.1145/2628136.2628138 [Accessed on: 2026 Jan 21]
4. David Emery. Standards, APIs, Interfaces and Bindings. 2015 Jan 16. Available from: https://web.archive.org/web/20150116081559/http://www.acm.org/tsc/apis.html [Accessed on: 2026 Jan 22]
5. Willis J, Wu N, and Pickering M. Staged selective parser combinators. Proceedings of the ACM on Programming Languages. 2020 Aug 2; 4(ICFP):1–30. DOI: 10.1145/3409002. Available from: https://dl.acm.org/doi/10.1145/3409002 [Accessed on: 2026 Jan 14]
6. Pickering M, Wu N, and Kiss C. Multi-stage programs in context. *Proceedings of the 12th ACM SIGPLAN International Symposium on Haskell*. ICFP '19: ACM SIGPLAN International Conference on Functional Programming. Berlin Germany: ACM, 2019 Aug 8:71–84. DOI: 10.1145/3331545.3342597. Available from: https://dl.acm.org/doi/10.1145/3331545.3342597 [Accessed on: 2026 Jan 14]
7. Tomas Akenine-Möller, Eric Haines, and Naty Hoffman. Real-time Rendering. 3rd. 2008 Jul. 1045 pp. Available from: https://learning.oreilly.com/library/view/real-time-rendering-third/9781439865293/chapter-10.html [Accessed on: 2026 Jan 18]
8. Janičić P. Geometry Constructions Language. Journal of Automated Reasoning. 2010 Feb; 44:3–24. DOI: 10.1007/s10817-009-9135-8. Available from: http://link.springer.com/10.1007/s10817-009-9135-8 [Accessed on: 2026 Jan 14]

9. Seitz KA, Foley T, Porumbescu SD, and Owens JD. Staged metaprogramming for shader system development. ACM Transactions on Graphics. 2019 Dec 31; 38:1–15. DOI: 10.1145/3355089.3356554. Available from: https://dl.acm.org/doi/10.1145/3355089.3356554 [Accessed on: 2026 Jan 14]

10. Scholz D. A Modular Domain-Specific Language for Interactive 3D Visualization. 2021. In collab. with Gröller E. Artwork Size: 104 pages Medium: application/pdf:104 pages. DOI: 10.34726/HSS.2021.80484. Available from: https://repositum.tuwien.at/handle/20.500.12708/17807 [Accessed on: 2026 Jan 14]

11. Jr. Richard S. Wright, Nicholas Haemel, Graham Sellers, and Benjamin Lipchak. OpenGL SuperBible: Comprehensive Tutorial and Reference, Fifth Edition. 2010 Jul. 1008 pp. Available from: https://learning.oreilly.com/library/view/opengl-superbible-comprehensive/9780132160926/ [Accessed on: 2026 Jan 22]

12. Mark Segal and Kurt Akeley. The OpenGL® Graphics System: A Specification (Version 4.6 (Core Profile) - May 5, 2022). 2022 May 5

13. David Herberth. GLAD Wiki - GitHub. 2019 May 1. Available from: https://github.com/Dav1dde/glad/wiki/C [Accessed on: 2026 Jan 22]

14. Anton Gerdelan. "Hello Triangle" - Anton's OpenGL 4 Tutorials. 2025 Nov 12. Available from: https://antongerdelan.net/opengl/hellotriangle.html [Accessed on: 2026 Jan 22]

15. Kesson M. Pixar's RenderMan. *ACM SIGGRAPH ASIA 2008 courses*. SIGGRAPH Asia '08. New York, NY, USA: Association for Computing Machinery, 2008 Dec 10:1–138. DOI: 10.1145/1508044.1508096. Available from: https://dl.acm.org/doi/10.1145/1508044.1508096 [Accessed on: 2026 Jan 20]

16. Hanrahan P and Catmull E. The Design of RenderMan. IEEE Computer Graphics and Applications. 2021 Jul; 41:7–15. DOI: 10.1109/MCG.2021.3085374. Available from: https://ieeexplore.ieee.org/document/9487525/ [Accessed on: 2026 Jan 20]

17. Raghavachary S. A brief introduction to RenderMan. *ACM SIGGRAPH 2006 Courses*. SIGGRAPH '06. New York, NY, USA: Association for Computing Machinery, 2006 Jul 30:2–es. DOI: 10.1145/1185657.1185817. Available from: https://dl.acm.org/doi/10.1145/1185657.1185817 [Accessed on: 2026 Jan 20]

18. Tobias Bexelius. GPipe: GPipe is dead, long live GPipe! GPipe. 2015 Sep 1. Available from: https://tobbebex.blogspot.com/2015/09/gpipe-is-dead-long-live-gpipe.html [Accessed on: 2026 Jan 21]

19. Tobias Bexelius. GPipe: GPU programming in Haskell using GPipe - Part 1. GPipe. 2015 Sep 6. Available from: https://tobbebex.blogspot.com/2015/09/gpu-programming-in-haskell-using-gpipe.html [Accessed on: 2026 Jan 21]

20. McCabe T. A Complexity Measure. IEEE Transactions on Software Engineering. 1976 Dec; SE-2:308–20. DOI: 10.1109/TSE.1976.233837. Available from: https://ieeexplore.ieee.org/document/1702388 [Accessed on: 2026 Jan 22]